

Module 3 – SQL Query Analysis Results

Laila Afmege — JHED: lafmege1 — Grad Cafe Admissions Data (30,060 records)

Question 1

Question: How many entries in the database are from applicants who applied for Fall 2026?

Result: Fall 2026 applicant count: 29,637

SQL query:

```
SELECT count(*) FROM applicants WHERE term ILIKE 'Fall 2026';
```

Explanation: Counts every row whose term field matches 'Fall 2026', case-insensitively (ILIKE) so capitalization differences in the scraped data don't cause misses.

Question 2

Question: Among entries that provide a nationality classification, what percentage are international students?

Result: Percent international: 46.36%

SQL query:

```
SELECT round(
  100.0 * count(*) FILTER (WHERE lower(us_or_international) = 'international')
  / NULLIF(count(*) FILTER (WHERE us_or_international IS NOT NULL
    AND btrim(us_or_international) <> ''), 0)
, 2)
FROM applicants;
```

Explanation: The denominator counts every entry with a non-blank nationality classification (American, International, or Other); the numerator counts only International. NULLIF guards against a divide-by-zero if no rows had a usable classification.

Question 3

Question: What are the average GPA, GRE Quantitative, GRE Verbal, and GRE Analytical Writing scores of applicants who provide each metric?

Result: Average GPA: 3.77 | Average GRE Quantitative: 165.83 | Average GRE Verbal: 160.71 | Average GRE Analytical Writing: 4.35

SQL query:

```
SELECT
  round(avg(gpa)::numeric, 2) AS avg_gpa,
  round(avg(gre) FILTER (WHERE gre BETWEEN 130 AND 170)::numeric, 2) AS avg_gre_quant,
  round(avg(gre_v) FILTER (WHERE gre_v BETWEEN 130 AND 170)::numeric, 2) AS avg_gre_verbal,
  round(avg(gre_aw) FILTER (WHERE gre_aw BETWEEN 0 AND 6)::numeric, 2) AS avg_gre_aw
FROM applicants;
```

Explanation: Each average is computed independently over only the applicants who provided that metric (SQL's avg() already ignores NULLs, so an applicant missing GRE scores still contributes to the GPA average). The GRE Quantitative and Verbal averages are restricted to the valid ETS 130-170 scale: 1,416 of 2,373 raw 'GRE' values (60%) fell outside that range -- almost certainly combined scores, percentile ranks, or typos rather than genuine Quantitative scores. This is a real, disclosed data-quality decision, discussed further in limitations.pdf.

Question 4

Question: What is the average GPA of American applicants who applied for Fall 2026?

Result: Average GPA (American, Fall 2026): 3.79

SQL query:

```
SELECT round(avg(gpa)::numeric, 2)
FROM applicants
WHERE term ILIKE 'Fall 2026'
      AND lower(us_or_international) = 'american'
      AND gpa IS NOT NULL;
```

Explanation: Filters to Fall 2026, American applicants, and requires a non-null GPA before averaging, satisfying all three stated conditions simultaneously.

Question 5

Question: What percentage of Fall 2025 entries are acceptances?

Result: Fall 2025 acceptance percentage: 47.92%

SQL query:

```
SELECT round(
    100.0 * count(*) FILTER (WHERE lower(status) = 'accepted')
    / NULLIF(count(*), 0)
, 2)
FROM applicants
WHERE term ILIKE 'Fall 2025';
```

Explanation: The denominator is every Fall 2025 entry regardless of status; the numerator is the subset whose cleaned status is 'Accepted'.

Question 6

Question: What is the average GPA of accepted applicants who applied for Fall 2026?

Result: Average GPA (Accepted, Fall 2026): 3.76

SQL query:

```
SELECT round(avg(gpa)::numeric, 2)
FROM applicants
WHERE term ILIKE 'Fall 2026'
      AND lower(status) = 'accepted'
      AND gpa IS NOT NULL;
```

Explanation: Same pattern as Question 4, but filtered on acceptance status instead of nationality.

Question 7

Question: How many entries are from applicants who applied to Johns Hopkins University for a master's degree in Computer Science?

Result: JHU Masters Computer Science count: 8

SQL query:

```
SELECT count(*)
FROM applicants
WHERE (program ILIKE '%hopkins%' OR program ILIKE '%jhu%')
      AND program ILIKE '%computer science%'
      AND lower(degree) = 'masters';
```

Explanation: program stores the combined 'Department/Program, University' string (the same original downloaded field convention used throughout Module 2). Matching '%hopkins%' catches 'Johns Hopkins University', 'JHU', and the 'John Hopkins' typo variant seen in the raw data; degree is normalized to the literal value 'Masters' by clean.py.

Question 8

Question: How many Fall 2026 entries are acceptances from applicants applying for a PhD in Computer Science at Georgetown, MIT, Stanford, or Carnegie Mellon?

Result: Original-field count: 28

SQL query:

```
SELECT count(*)
FROM applicants
WHERE term ILIKE 'Fall 2026'
      AND lower(status) = 'accepted'
      AND lower(degree) = 'phd'
      AND program ILIKE '%computer science%'
      AND (
        program ILIKE '%georgetown%'
        OR program ILIKE '%massachusetts institute of technology%'
        OR program ~* '\ymit\y'
        OR program ILIKE '%stanford%'
        OR program ILIKE '%carnegie mellon%'
      );
```

Explanation: All five conditions (term, status, degree, subject, and one of the four universities) apply simultaneously via AND. MIT is matched with a word-boundary regex (\y...y) rather than a plain substring, since a plain '%mit%' match would also catch unrelated programs like 'Committee on Evolutionary Biology' -- confirmed by inspecting the actual data before finalizing this query.

Question 9

Question: Repeat Question 8 using llm_generated_program / llm_generated_university instead of the original downloaded fields.

Result: Original-field count: 28 | LLM-field count: 0 | Difference: -28

SQL query:

```
SELECT count(*)
FROM applicants
WHERE term ILIKE 'Fall 2026'
      AND lower(status) = 'accepted'
      AND lower(degree) = 'phd'
      AND llm_generated_program ILIKE '%computer science%'
      AND (
        llm_generated_university ILIKE '%georgetown%'
        OR llm_generated_university ILIKE '%massachusetts institute of technology%'
        OR llm_generated_university ~* '\ymit\y'
        OR llm_generated_university ILIKE '%stanford%'
        OR llm_generated_university ILIKE '%carnegie mellon%'
      );
```

Explanation: term/degree/status still come from the original fields as instructed; only the subject/university matching switches to the LLM-generated fields. The LLM-field count is lower for a specific, documented reason: the local LLM standardization pass (Module 2) only finished processing 60 of 30,060 cleaned records before the deadline, so llm_generated_program/university are NULL for the other 29,940 rows. This is a coverage gap in how much data was standardized, not evidence that the LLM's matching disagrees with the original fields -- with

the full run complete, we would expect the two counts to converge closely, since the LLM was only asked to standardize spelling/naming, not to reinterpret program or university identity.

Original Question 1

Question: Does a higher undergraduate GPA correlate with a higher acceptance rate?

Result: GPA < 3.8: 6,927 entries, 43.74% accepted | GPA >= 3.8: 11,136 entries, 39.29% accepted

SQL query:

```
SELECT
  CASE WHEN gpa >= 3.8 THEN 'GPA >= 3.8' ELSE 'GPA < 3.8' END AS gpa_bucket,
  count(*) AS total,
  round(100.0 * count(*) FILTER (WHERE lower(status) = 'accepted') / count(*), 2) AS pct_accepted
FROM applicants
WHERE gpa IS NOT NULL AND status IS NOT NULL
GROUP BY 1
ORDER BY 1;
```

Explanation: Buckets every applicant with both a GPA and a decision into two groups and computes each group's acceptance rate. Counterintuitively, the higher-GPA group was accepted slightly less often -- most likely because applicants with stronger GPAs disproportionately apply to more selective (and more competitive) PhD programs, not because a higher GPA hurts admission odds. This finding is discussed further in [limitations.pdf](#) as an example of why raw correlations in self-submitted data can be misleading without controlling for program selectivity.

Original Question 2

Question: How many entries are there for each degree type, and which is the most common?

Result: PhD: 21,243 | Masters: 7,640 | MFA: 621 | PsyD: 301 | Other: 211 | JD: 18 | EdD: 17 | MBA: 9

SQL query:

```
SELECT degree, count(*) AS total
FROM applicants
WHERE degree IS NOT NULL
GROUP BY degree
ORDER BY total DESC;
```

Explanation: Groups all entries by their normalized degree type and counts each group, ordered from most to least common. PhD applications dominate the dataset (over 70% of all entries), which is itself a useful representativeness signal discussed in [limitations.pdf](#).